Example 1. Determine the expected running time of the given algorithm. Assume that `Random`(n) generates a random integer between 1 and n with uniform distribution.

```
1:  function Func7(A[],n)
2:      if n ≤ 2 then
3:          return A[1]
4:      else
5:          s = 0
6:          for i = 1 to n − 1 do
7:              A[i] = A[i] + A[i + 1]
8:              s = s + A[i]
9:          end for
10:         k = Random(n − 1)
11:         s = Func7(A[], k)
12:         return s
13:     end if
14: end function
```

We first note that this algorithm is notably different, because the recursive call on line 11 has a **random size**, therefore we must necessarily adapt our analysis to work with this!

Best case Let $T_b(n)$ be the best case running time. The best case occurs when $k = 1$ and obeys the recurrence relation $T_b(n) = c(n-1) + T_b(1) = c(n-1) + c = cn$. So, the best case running time is in $\Theta(n)$.

Worst case Let $T_w(n)$ be the worst case running time. The worst case occurs when $k = n-1$ and obeys the recurrence relation $T_w(n) = c(n-1) + T_w(n-1)$ Therefore, the worst case running time is in $\Theta(n^2)$.

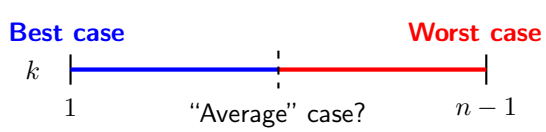
Expected case Let $ET(n)$ be the expected running time of `Func7` with an input of size n. Suppose we attempted to analyze this problem using the definition of expected run time,

$$ET(n) = \sum_{q=1}^{n-1} \textbf{Prob}\,(k = q)\, t(k = q)$$

One of these terms is quite convenient, since $\textbf{Prob}\,(k = q) = \frac{1}{n-1}$, however $t(k = q) = c(n-1) + ET(q)$ which may be problematic,

$$ET(n) = \sum_{q=1}^{n-1} \frac{1}{n-1}(c(n-1) + ET(q)) = c(n-1) + \frac{1}{n-1}\sum_{q=1}^{n-1} ET(q).$$

This summation represents $n-1$ recursive calls each with a likelihood of $\frac{1}{n-1}$ – it turns out that analyzing such a summation becomes extremely difficult. Instead of using this methodology, we shall try and leverage conditional expectation to make the problem more tractable. We begin by visualizing the range of values for k,



As we can see in our visualization, our aim is to "split" the problem between the best case and worst case. One "convenient" value between the best case and the worst case is $\frac{n}{2}$, so we write the expected running time as

$$ET(n) = E\left(T(n) \mid k < \frac{n}{2}\right) \textbf{Prob}\left(k < \frac{n}{2}\right) + E\left(T(n) \mid k \geq \frac{n}{2}\right) \textbf{Prob}\left(k \geq \frac{n}{2}\right). \tag{1}$$

We observe that diving the problem as such means that the probabilities involved have become easier to contend with since **Prob** $\left(k < \frac{n}{2}\right) \approx \frac{1}{2}$ and **Prob** $\left(k \geq \frac{n}{2}\right) \approx \frac{1}{2}$. Furthermore, we can now **bound** the size of the recursive call within each range of values. The largest recursive call possible when $k < \frac{n}{2}$ is $ET(\frac{n}{2})$ and the largest recursive call possible when $k \geq \frac{n}{2}$ is $ET(n-1)$. Therefore,

$$\begin{aligned}
E\left(T(n) \mid k < \frac{n}{2}\right) &\leq c(n-1) + ET\left(\frac{n}{2}\right) \\
E\left(T(n) \mid k \geq \frac{n}{2}\right) &\leq c(n-1) + ET\left(n-1\right)
\end{aligned}$$

So, using all of this information in conjunction with Equation (1) we have

$$\begin{aligned}
ET(n) &= E\left(T(n) \mid k < \frac{n}{2}\right) \textbf{Prob}\left(k < \frac{n}{2}\right) + E\left(T(n) \mid k \geq \frac{n}{2}\right) \textbf{Prob}\left(k \geq \frac{n}{2}\right). \\
ET(n) &\leq \left(c(n-1) + ET\left(\frac{n}{2}\right)\right) \cdot \frac{1}{2} + \left(c(n-1) + ET\left(n-1\right)\right) \cdot \frac{1}{2} \\
ET(n) &\leq c(n-1) + \frac{1}{2}ET\left(\frac{n}{2}\right) + \frac{1}{2}ET(n-1).
\end{aligned}$$

Next we observe that since $ET(n)$ is a non-decreasing function, $ET(n-1) \leq ET(n)$,

$$\begin{aligned}
ET(n) &\leq c(n-1) + \frac{1}{2}ET\left(\frac{n}{2}\right) + \frac{1}{2}ET(n-1), && \text{Since } ET(n-1) \leq ET(n) \\
ET(n) &\leq c(n-1) + \frac{1}{2}ET\left(\frac{n}{2}\right) + \frac{1}{2}ET(n), && \text{Rearrange to group together } ET(n) \text{ terms} \\
ET(n) - \frac{1}{2}ET(n) &\leq c(n-1) + ET\left(\frac{n}{2}\right), \\
\frac{1}{2}ET(n) &\leq c(n-1) + ET\left(\frac{n}{2}\right), && \text{Multiply by 2 and let } c_1 = 2c \\
ET(n) &\leq c_1(n-1) + ET\left(\frac{n}{2}\right) \\
ET(n) &\leq c_1 n + ET\left(\frac{n}{2}\right).
\end{aligned}$$

Now, we have a very standard recurrence relation to analyze! So,

$$\begin{aligned}
ET(n) &\leq c_1 n + ET\left(\frac{n}{2}\right) \\
ET(n) &\leq c_1 n + \frac{c_1 n}{2} + ET\left(\frac{n}{2^2}\right) \\
ET(n) &\leq c_1 n + \frac{c_1 n}{2} + \frac{c_1 n}{2^2} + ET\left(\frac{n}{2^3}\right) \\
ET(n) &\leq \sum_{i=0}^{k-1} \frac{c_1 n}{2^i} + ET\left(\frac{n}{2^k}\right)
\end{aligned}$$

We choose k such that

$$\frac{n}{2^k} = 1 \qquad \Longrightarrow \qquad k = \log_2(n),$$

so

$$
\begin{aligned}
ET(n) &\leq c_1 n \sum_{i=0}^{k-1} \frac{1}{2^i} + ET\left(1\right) \\
ET(n) &\leq c_1 n \sum_{i=0}^{\infty} \left(\frac{1}{2}\right) + c \\
ET(n) &\leq c_1 n \left(\frac{1}{1-\frac{1}{2}}\right) + c.
\end{aligned}
$$

Thus, $ET(n) \in O(n)$. Now, we can **always** bound the expected time below by best case running time. So, since the best case running time is in $\Theta(n)$, the expected case must be in $\Omega(n)$. Therefore,

$$\boxed{ET(n) \in \Theta(n).}$$

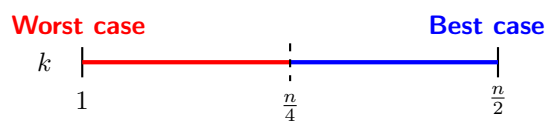
Example 2. Determine the expected running time of the given algorithm. Assume that `Random`(n) generates a random integer between 1 and n with uniform distribution.

```
function Func8(A[],n)
    if n ≤ 2 then
        return A[1]
    else
        s = 0
        for i = 1 to n − 1 do
            A[i] = A[i] + A[i + 1]
            s = s + A[i]
        end for
        k = Random(⌊n/2⌋)
        s = Func8(A[], k)
        s = Func8(A[], n − k)
        return s
    end if
end function
```

Best case Let $T_b(n)$ be the best case running time. The best case occurs when $k = \frac{n}{2}$ and obeys the recurrence relation $T_b(n) = cn + 2T_b(n/2)$. So, the best case running time is in $\Theta(n \log_2(n))$.

Worst case Let $T_w(n)$ be the worst case running time. The worst case occurs when $k = 1$ and obeys the recurrence relation $T_w(n) = cn + T_w(1) + T_w(n-1)$ Therefore, the worst case running time is in $\Theta(n^2)$.

Expected case Let $ET(n)$ be the expected running time of `Func8` with an input of size n. We begin by visualizing the range of values for k,



As we can see in our visualization, our aim is to "split" the problem between the best case and worst case. One "convenient" value between the best case and the worst case is $\frac{n}{2}$, so we write the expected running time as

$$ET(n) = E\left(T(n) \mid k < \frac{n}{4}\right) \textbf{Prob}\left(k < \frac{n}{4}\right) + E\left(T(n) \mid k \geq \frac{n}{4}\right) \textbf{Prob}\left(k \geq \frac{n}{4}\right). \tag{2}$$

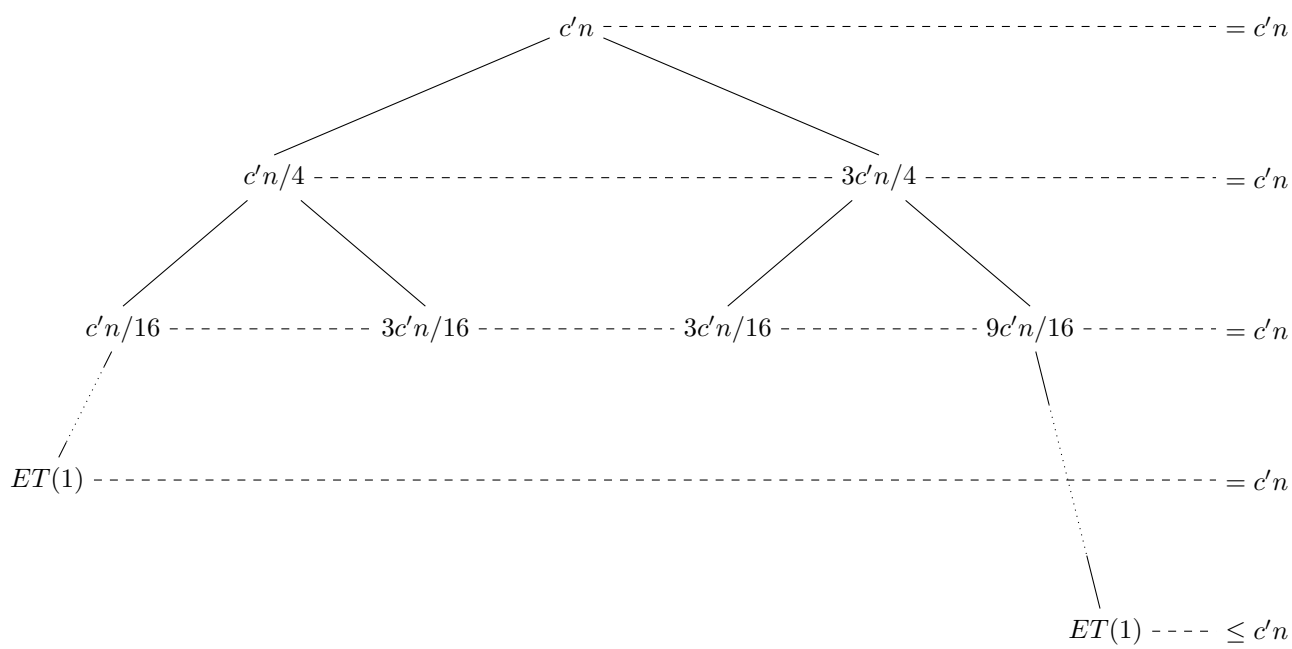
As before $\textbf{Prob}\left(k < \frac{n}{4}\right) = \frac{n/4}{n/2} = \frac{1}{2}$ and $\textbf{Prob}\left(k \geq \frac{n}{4}\right) = 1 - \textbf{Prob}\left(k < \frac{n}{4}\right) = \frac{1}{2}$. And again we shall attempt to bound size of the recursive calls within each range of values. So,

$$E\left(T(n) \mid k < \frac{n}{4}\right) \leq c(n-1) + ET(1) + ET(n-1) = cn + ET(n-1)$$
$$E\left(T(n) \mid k \geq \frac{n}{4}\right) \leq cn + ET\left(\frac{n}{4}\right) + ET\left(\frac{3n}{4}\right)$$

Using these bounds in Equation (2) we have

$$
\begin{aligned}
ET(n) &= E\left(T(n) \mid k < \frac{n}{4}\right) \textbf{Prob}\left(k < \frac{n}{4}\right) + E\left(T(n) \mid k \geq \frac{n}{4}\right) \textbf{Prob}\left(k \geq \frac{n}{4}\right) \\
ET(n) &\leq (cn + ET(n-1)) \cdot \frac{1}{2} + \left(cn + ET\left(\frac{n}{4}\right) + ET\left(\frac{3n}{4}\right)\right) \cdot \frac{1}{2} \\
ET(n) &\leq cn + \frac{1}{2}ET(n-1) + \frac{1}{2}ET\left(\frac{n}{4}\right) + \frac{1}{2}ET\left(\frac{3n}{4}\right) \\
ET(n) &\leq cn + \frac{1}{2}ET(n) + \frac{1}{2}ET\left(\frac{n}{4}\right) + \frac{1}{2}ET\left(\frac{3n}{4}\right) \\
\frac{1}{2}ET(n) &\leq cn + \frac{1}{2}ET\left(\frac{n}{4}\right) + \frac{1}{2}ET\left(\frac{3n}{4}\right) \\
ET(n) &\leq 2cn + ET\left(\frac{n}{4}\right) + ET\left(\frac{3n}{4}\right) \\
ET(n) &\leq c'n + ET\left(\frac{n}{4}\right) + ET\left(\frac{3n}{4}\right)
\end{aligned}
$$

We have seen similar recurrence relations in the past and analyzed it using a recursion tree. We proceed likewise,



We need only bound above so we must determine the height of this tree. We can find the height by solving

$$\frac{3^h n}{4^h} = 1 \qquad \Longrightarrow \qquad h = \log_{4/3}(n)$$

know that each level's cost is bounded by $c'n$ and the height of the tree, therefore,

$$ET(n) \leq c'n \log_{4/3}(n).$$

Thus, $ET(n) \in O(n \log(n))$. As with our previous example, the best case running time always provides a

lower bound on the expected case, so $ET(n) \in \Omega(n \log(n))$. Therefore,

$$\boxed{ET(n) \in \Theta(n \log(n)).}$$

Example 3. Determine the expected running time of the given algorithm.

```
1: function Func9()
2:     repeat
3:         c = CoinFlip()
4:         if c = heads then
5:             Print "I win"
6:         else
7:             Print "I quit"
8:         end if
9:     until c = tails
10: end function
```

This example is our first instance of a "repeat-until" loop structure, which is essentially a "do-while" loop. The code block inside of the repeat-until loop executes until the condition in the "until" clause is satisfied. Notably, this loop does not have a maximum number of iterations!

Best case The best case running time for this algorithm is when a "tails" is flipped on the first iteration. In this case, the algorithm takes $\Theta(1)$ time.

Worst case The worst case running time for this algorithm occurs when a "tails" is **never** flipped. In this case, the algorithm takes $O(\infty)$ time, which last I checked is not the best running time.

Expected case Let $T(n)$ be the running time of `Func9`. To analyze the expected case running time, we need first define X to be the number of times that "heads" is flipped on line 3. Having defined this quantity, we can express the running time as

$$T(n) = c + cX$$

We can find the expected value of this expression (and use linearity) to express $ET(n)$ as

$$ET(n) = c + cE(x)$$

We have now reduced the problem to a probability problem: namely, what is the expected number of heads flipped on line 3? Analyzing this by definition can be rather tricky,

$$E(X) = \sum_{i=0}^{\infty} i\textbf{Prob}\left(X = i\right)$$

so we instead use the formula for non-negative probabilities that states

$$E(X) = \sum_{i=1}^{\infty} \textbf{Prob}\left(X \geq i\right)$$

and, conveniently, **Prob** $(X \geq i)$ is "the probability of getting at least i heads." So,

$$\begin{aligned}
\textbf{Prob}\,(X \geq 1) &= \frac{1}{2} \\
\textbf{Prob}\,(X \geq 2) &= \frac{1}{2} \cdot \frac{1}{2} = \left(\frac{1}{2}\right)^2 \\
\textbf{Prob}\,(X \geq 3) &= \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = \left(\frac{1}{2}\right)^3 \\
\textbf{Prob}\,(X \geq i) &= \left(\frac{1}{2}\right)^i .
\end{aligned}$$

Therefore,

$$E(X) = \sum_{i=1}^{\infty} \textbf{Prob}\,(X \geq i) = \sum_{i=1}^{\infty} \left(\frac{1}{2}\right)^i = \frac{\frac{1}{2}}{1 - \frac{1}{2}} = 1$$

Thus,

$$ET(n) = c + cE(X) = 2c$$

So,

$$\boxed{ET(n) \in \Theta(1).}$$

Example 4. Determine the expected running time of the given algorithm.

```
function Func10()
    repeat
        d = RollD6()
        if d = 6 then
            Print "I quit"
        else
            Print "I win one dollar"
        end if
    until d = 6
end function
```

Best case The best case running time for this algorithm is when a 6 is rolled on the first iteration. In this case, the algorithm takes $\Theta(1)$ time.

Worst case The worst case running time for this algorithm occurs when a 6 is **never** rolled. In this case, the algorithm takes $O(\infty)$ time.

Expected case Let $T(n)$ be the running time of `Func10`. Let X to be the number of times that a 6 is not rolled. Having defined this quantity, we can express the running time as

$$T(n) = c + cX$$

We can find the expected value of this expression (and use linearity) to express $ET(n)$ as

$$ET(n) = c + cE(X)$$

We next use the formula for non-negative probabilities that states

$$E(X) = \sum_{i=1}^{\infty} \textbf{Prob}\,(X \geq i)$$

and, conveniently, **Prob** $(X \geq i)$ is "the probability of not rolling a 6 i times" So,

$$\textbf{Prob}\,(X \geq i) = \left(\frac{5}{6}\right)^i.$$

Therefore,

$$E(X) = \sum_{i=1}^{\infty} \textbf{Prob}\,(X \geq i) = \sum_{i=1}^{\infty} \left(\frac{5}{6}\right)^i = \frac{\frac{5}{6}}{1 - \frac{5}{6}} = 5$$

Thus,

$$ET(n) = c + cE(X) = 6c$$

So,

$$\boxed{ET(n) \in \Theta(1).}$$

Example 5. Determine the expected running time of the given algorithm.

```
1: function Func11(A[],n)
2:     for i = 1 to n do
3:         d_1 = RollD6()
4:         d_2 = RollD6()
5:         if d_1 = 1 and d_2 = 1 then
6:             return A[i]
7:         end if
8:     end for
9: end function
```

Best case The best case running time for this algorithm is when $d_1 = d_2 = 1$ on the first iteration. In this case, the algorithm takes $\Theta(1)$ time.

Worst case The worst case running time for this algorithm occurs $d_1 \neq 1$ or $d_2 \neq 1$ every iteration. In this case, the algorithm takes $O(\infty)$ time.

Expected case Let $T(n)$ be the running time of `Func11`. Let X to be the number of times the return statement on line 6 is skipped. We can express the running time as

$$T(n) = c + cX$$

We can find the expected value of this expression (and use linearity) to express $ET(n)$ as

$$ET(n) = c + cE(X)$$

We next use the formula for non-negative probabilities that states

$$E(X) = \sum_{i=1}^{\infty} \textbf{Prob}\,(X \geq i)$$

and since $\textbf{Prob}\,(d_1 = 1 \text{ and } d_2 = 1) = \frac{1}{36}$,

$$\textbf{Prob}\,(X \geq 1) = \frac{35}{36}$$
$$\textbf{Prob}\,(X \geq i) = \left(\frac{35}{36}\right)^i.$$

Therefore,

$$E(X) = \sum_{i=1}^{\infty} \textbf{Prob}\,(X \geq i) = \sum_{i=1}^{\infty} \left(\frac{35}{36}\right)^i = \frac{\frac{35}{36}}{1 - \frac{35}{36}} = 35$$

Thus,

$$ET(n) = c + cE(X) = 36c$$

So,

$$\boxed{ET(n) \in \Theta(1).}$$

Example 6. Determine the expected running time of the given algorithm.

```
function Func12(m)
    B is a bin of m red balls and 2m green balls.
    repeat
        β =select and remove a random ball from B
        if β is red then
            Print "I quit"
        else
            Print "I win one dollar"
        end if
    until β is red
end function
```

Remark: This code, it turns out, is **very** similar to the code for open address hashing which we will analyze later in the course!

Best case The best case running time for this algorithm is when the first ball removed is red. In this case, the algorithm takes $\Theta(1)$ time.

Worst case The worst case running time for this algorithm occurs we remove all $2m$ green balls before removing a red ball. In this case, the algorithm takes $\Theta(m)$ time.

Expected case Let $T(n)$ be the running time of `Func12` with input m. Let X to be the number of green balls selected and removed. We can express the running time as

$$T(n) = c + cX$$

We can find the expected value of this expression (and use linearity) to express $ET(n)$ as

$$ET(n) = c + cE(X)$$

We next use the formula for non-negative probabilities that states

$$E(X) = \sum_{i=1}^{\infty} \textbf{Prob}\left(X \geq i\right).$$

Unfortunately, computing $\textbf{Prob}\left(X \geq i\right)$ is not as easy as it was in our previous examples due to the balls being removed each iteration. First, we observe that

$$\textbf{Prob}\left(X \geq 1\right) = \frac{2m}{3m}$$

To compute the next probability we observe that for $X \geq 2$ to be possible, we must have selected a green ball on the first draw and removed it from the bin and have done the same on the second draw The total number of green balls present for the second draw would be $2m - 1$ and total number of balls would be $3m - 1$, therefore

$$\textbf{Prob}\left(X \geq 2\right) = \frac{2m}{3m} \cdot \frac{2m-1}{3m-1}.$$

Using similar reasoning we have

$$\textbf{Prob}\,(X \geq 3) = \frac{2m}{3m} \cdot \frac{2m-1}{3m-1} \cdot \frac{2m-2}{3m-2}.$$

So, in general,

$$\textbf{Prob}\,(X \geq i) = \frac{2m}{3m} \cdot \frac{2m-1}{3m-1} \cdots \frac{2m-i+1}{3m-i+1}.$$

This quantity looks *very* difficult to deal with, so we will attempt to bound it by a more convenient expression. First, we observe that

$$\frac{x-1}{y-1} \leq \frac{x}{y}$$

so long as $\frac{x}{y} \leq 1$ and $y > 1$. Importantly,

$$\frac{2m-i+1}{3m-i+1} \leq \cdots \leq \frac{2m-2}{3m-2} \leq \frac{2m-1}{3m-1} \leq \frac{2m}{3m}$$

so

$$\textbf{Prob}\,(X \geq i) = \frac{2m}{3m} \cdot \frac{2m-1}{3m-1} \cdots \frac{2m-i+1}{3m-i+1} \leq \frac{2m}{3m} \cdot \frac{2m}{3m} \cdots \frac{2m}{3m} = \left(\frac{2}{3}\right)^i.$$

and $\textbf{Prob}\,(X \geq i) = 0$ when $i > 2m$. Bringing all of this together,

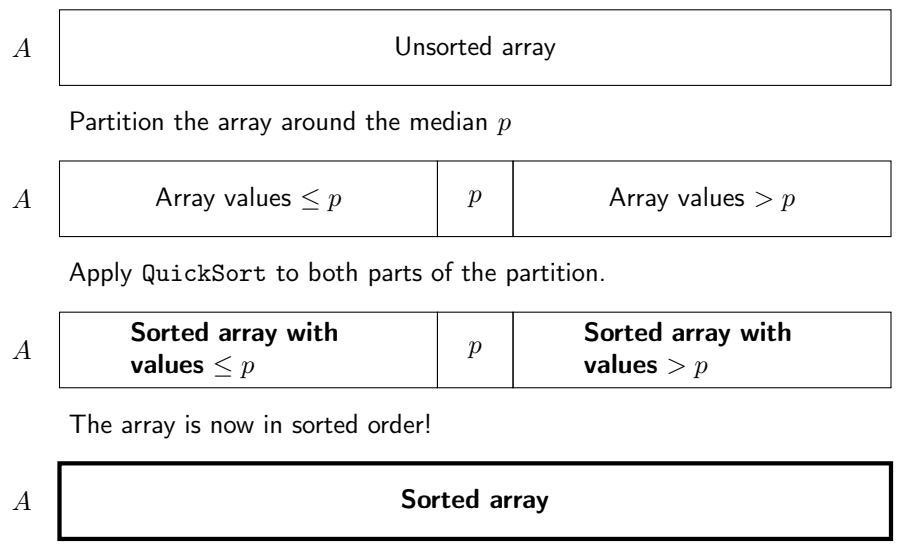
$$ET(n) = c + cE(x) = c + c\sum_{i=1}^{\infty} \textbf{Prob}\,(X \geq i) = c + c\sum_{i=1}^{2m} \left(\frac{2}{3}\right)^i \leq c + c\frac{\frac{2}{3}}{1-\frac{2}{3}} = 3c$$

Therefore,

$$\boxed{ET(n) \in \Theta(1).}$$

```
function QuickSort1(A[],i,j)
    if i < j then
        p = Median(A, i, j)
        ▷ Partition A[i..j] using p such that A[s] = p
        ▷ and A[i'] ≤ p ≤ A[j'] for i ≤ i' ≤ s ≤ j' ≤ j
        s = Partition(A, i, j, p)
        QuickSort1(A[],i,s − 1)
        QuickSort1(A[],s + 1,j)
    end if
end function
```

This implementation of `QuickSort` sorts the array $A[i..j]$ of size $n = j - i + 1$ by **partitioning** the array about the value and sorting both parts of the partition recursively.



Assume that `Median` takes $\Theta(n)$ time[1] and that `Partition` takes $\Theta(n)$ time (we will prove this eminently), then this algorithm obeys

$$T(n) = cn + 2T(n/2) \qquad T(1) = c$$

which means that $T(n) \in \Theta(n \log(n))$. While this algorithm seems convenient, the implementation of `Median` in linear time takes a good deal of storage and does not improve on the asymptotic runtime of `MergeSort`. We can lower the spatial complexity (the amount of additional storage required to run the algorithm) by introducing some randomness into the algorithm. This randomized implementation of `QuickSort` is the more standard version of the algorithm.

[1] We will discuss later how this is possible in the expected case. The CLRS book explains an algorithm that can do this in worst case $\Theta(n)$, but we will not cover it in these notes

```
function Partition(A[],i,j,p)
    low = i
    high = j
    while low < high do
        if A[low] < p then
            low = low + 1
        else if A[high] > p then
            high = high − 1
        else
            Swap(A[low],A[high])
            if A[low] = p and A[high] = p then
                low = low + 1
            end if
        end if
    end while
end function
```

Define $n = j - i + 1$. On each iteration, either

1. low is increased by 1.
2. $high$ is decreased by 1.
3. $A[low]$ and $A[high]$ are swapped and on the next iteration low is increased by 1.
4. $A[low]$ and $A[high]$ are swapped and low is increased by 1.

Therefore, the maximum number of iterations of the while loop in `Partition` is $2n$ since the total number of elements "out of order" is reduced by at least one ever other iteration. Each iteration takes constant time, so `Partition` takes $\Theta(n)$ time in best, worst, and expected case.

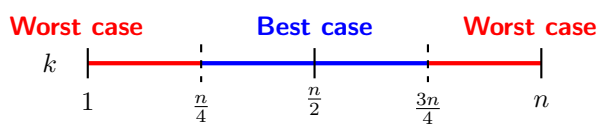
```
1: function QuickSort(A[],i,j)
2:     if i < j then
3:         p = RandomElement(A, i, j)
           ▷ Partition A[i..j] using p such that A[s] = p
           ▷ and A[i'] ≤ p ≤ A[j'] for i ≤ i' ≤ s ≤ j' ≤ j
4:         s = Partition(A, i, j, p)
5:         QuickSort(A[],i,s − 1)
6:         QuickSort(A[],s + 1,j)
7:     end if
8: end function
```

Let $T(n)$ be the the running time of `QuickSort` with input of size $n = j + i - 1$. We shall first try and make this problem a little more familiar by defining $k = s - i + 1$; this corresponds to the relative position of the pivot in the array. This algorithm sorts the array between positions i and j, so $s \in [i, j]$, but $k \in [1, n]$ which is similar to what we saw in our previous examples. The two recursive calls made on lines 5 and 6 are of size $s - i = k - 1$ and $j - s = j - (k + i - 1) = j - i + 1 - k = n - k$. This problem is now very reminiscent of the problems discussed earlier in these notes and we shall analyze it as such.

Best case The best case occurs when the array is evenly partitioned – that is, $k = \frac{n}{2}$. In this case the algorithm obeys $T(n) = cn + T(\frac{n}{2} - 1) + T(\frac{n}{2}) \approx cn + 2T(\frac{n}{2})$ which means the best case running time is $\Theta(n \log(n))$.

Worst case The worst case occurs when the array is as unevenly partitioned as possible – that is $k = 1$ **or** $k = n$. In this case the algorithm obeys $T(n) = cn + T(1) + T(n - 1)$ so the worst case running time is $\Theta(n^2)$.

Expected case For the expected case we shall, as before, visualize the range of values of k,



We can now express the running time as

$$ET(n) = E\left(T(n) \mid k \le \frac{n}{4}\right) \textbf{Prob}\left(k \le \frac{n}{4}\right) + E\left(T(n) \mid \frac{n}{4} < k < \frac{3n}{4}\right) \textbf{Prob}\left(\frac{n}{4} < k < \frac{3n}{4}\right) + E\left(T(n) \mid \frac{3n}{4} \le k \le n\right) \textbf{Prob}\left(\frac{3n}{4} \le k \le n\right)$$

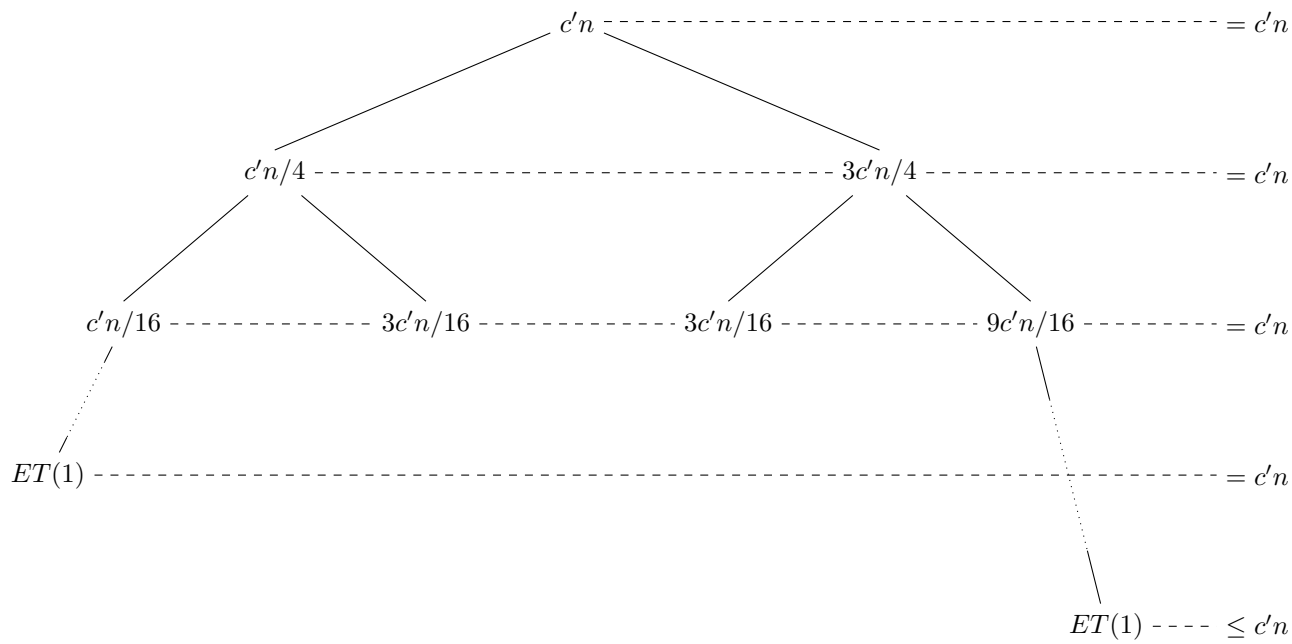
The random element chosen on line 3 is equally likely to be in all positions in A, therefore **Prob** $\left(k \le \frac{n}{4}\right) \approx \frac{1}{4}$, **Prob** $\left(\frac{n}{4} < k < \frac{3n}{4}\right) \approx \frac{1}{2}$, **Prob** $\left(\frac{3n}{4} \le k \le n\right) \approx \frac{1}{4}$. Furthermore, our visualization helps us understand how to correctly bound each of the recursions. We always choose "the worst possible value of k" within the relevant range of values. So,

$$E\left(T(n) \mid k \le \frac{n}{4}\right) \le cn + ET(0) + ET(n-1) = cn + ET(n-1)$$
$$E\left(T(n) \mid \frac{n}{4} < k < \frac{3n}{4}\right) \le cn + ET\left(\frac{n}{4}\right) + ET\left(\frac{3n}{4}\right)$$
$$E\left(T(n) \mid \frac{3n}{4} \le k \le n\right) \le cn + ET(n-1) + ET(0) = cn + ET(n-1)$$

Bring all of this together we have

$$
\begin{aligned}
ET(n) &\leq (cn + ET(n-1)) \cdot \frac{1}{4} + \left(cn + ET\left(\frac{n}{4}\right) + ET\left(\frac{3n}{4}\right)\right) \cdot \frac{1}{2} + (cn + ET(n-1)) \cdot \frac{1}{4} \\
ET(n) &\leq cn + \frac{1}{2}ET(n-1) + \frac{1}{2}ET\left(\frac{n}{4}\right) + \frac{1}{2}ET\left(\frac{3n}{4}\right) \\
ET(n) &\leq cn + \frac{1}{2}ET(n) + \frac{1}{2}ET\left(\frac{n}{4}\right) + \frac{1}{2}ET\left(\frac{3n}{4}\right) \\
\frac{1}{2}ET(n) &\leq cn + \frac{1}{2}ET\left(\frac{n}{4}\right) + \frac{1}{2}ET\left(\frac{3n}{4}\right) \\
ET(n) &\leq c'n + ET\left(\frac{n}{4}\right) + ET\left(\frac{3n}{4}\right)
\end{aligned}
$$

Which is the **exact** same recurrence relation as we had before that we analyzed with a recursion tree. So,



We need only bound above so we must determine the height of this tree. We can find the height by solving

$$\frac{3^h n}{4^h} = 1 \qquad \Longrightarrow \qquad h = \log_{4/3}(n)$$

know that each level's cost is bounded by $c'n$ and the height of the tree, therefore,

$$ET(n) \leq c'n \log_{4/3}(n).$$

Thus, $ET(n) \in O(n \log(n))$. As with our previous example, the best case running time always provides a lower bound on the expected case, so $ET(n) \in \Omega(n \log(n))$. Therefore,

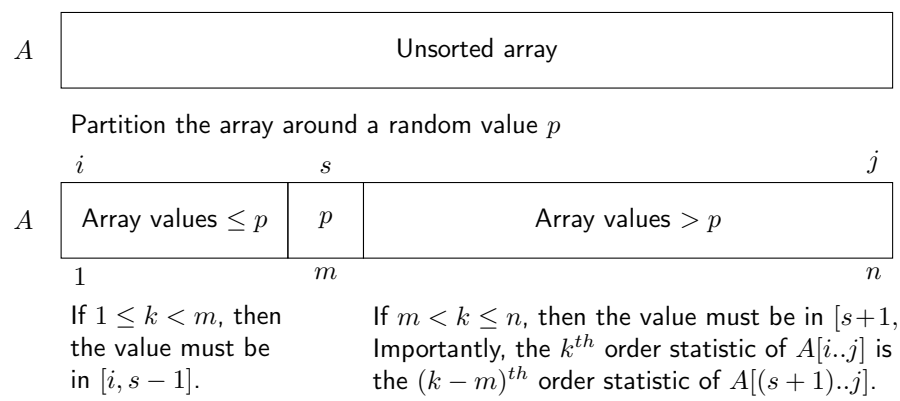
$$\boxed{ET(n) \in \Theta(n \log(n)).}$$

```
function QuickSort2(A[],i,j)
    if i < j then
        p = A[i]
        ▷ Partition A[i..j] using p such that A[s] = p
        ▷ and A[i'] ≤ p ≤ A[j'] for i ≤ i' ≤ s ≤ j' ≤ j
        s = Partition(A, i, j, p)
        QuickSort(A[],i,s − 1)
        QuickSort(A[],s + 1,j)
    end if
end function
```

Definition 1. The kth **order statistic** of an array is the kth smallest value.

```
function RandomizedSelect(A[],i,j,k)
    p = RandomElement(A, i, j)
    s = Partition(A[], i, j, p)
    m = s − i + 1
    if k = m then
        x = A[s]
    else if k < m then
        x = RandomizedSelect(A[], i, s − 1, k)
    else        ▷ k > m
        x = RandomizedSelect(A[], s + 1, j, k − m)
    end if
    return x
end function
```

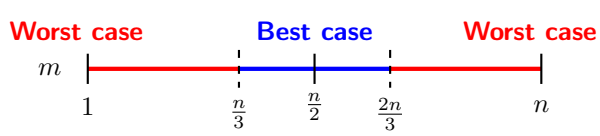
This randomized selection algorithm finds the kth order statistic by picking a random element, partitioning the array about that element, and then repeating the process on the "correct side" of the partition.



Best case The best case running time of `RandomizedSelect` occurs when $m = k$ on the first call of the function. In this case the algorithm takes $\Theta(n)$ time, because `Partition` takes $\Theta(n)$ time.

Worst case The worst case running time occurs on many different inputs, but one input is: If $k = 1$ and $m = j$ on every recursive call. In this case, the recurrence relation describing the worst case running time would be $T_w(n) = cn + T_w(n-1)$ which means that the worst case running time is $\Theta(n^2)$.

Expected case We shall focus our analysis on finding the median, though the analysis here could be generalized to finding any order statistic. That is, assume that $k = \frac{n}{2}$; as we have several times let us visualize the range of values of m (the random variable present in this algorithm),



Here we have made a different split than we did in `QuickSort` – we do this to emphasize that "splitting" the problem can be done arbitrarily! So,

$$ET(n) = E\left(T(n) \mid m \leq \frac{n}{3}\right) \textbf{Prob}\left(m \leq \frac{n}{3}\right) + E\left(T(n) \mid \frac{n}{3} < m < \frac{2n}{3}\right) \textbf{Prob}\left(\frac{n}{3} < m < \frac{2n}{3}\right) + E\left(T(n) \mid \frac{2n}{3} \leq m \leq n\right) \textbf{Prob}\left(\frac{2n}{3} \leq m \leq n\right)$$

We first observe that $\textbf{Prob}\left(m \leq \frac{n}{3}\right) = \frac{1}{3}$, $\textbf{Prob}\left(\frac{n}{3} < m < \frac{2n}{3}\right) = \frac{1}{3}$, and $\textbf{Prob}\left(\frac{2n}{3} \leq m \leq n\right) = \frac{1}{3}$ and we can bound each of the conditional expectations,

$$\begin{aligned} E\left(T(n) \mid m \leq \frac{n}{3}\right) &\leq cn + ET(n-1) \\ E\left(T(n) \mid \frac{n}{3} < m < \frac{2n}{3}\right) &\leq cn + ET\left(\frac{2n}{3}\right) \\ E\left(T(n) \mid \frac{2n}{3} \leq m \leq n\right) &\leq cn + ET(n-1). \end{aligned}$$

So,

$$\begin{aligned} ET(n) &\leq (cn + ET(n-1)) \cdot \frac{1}{3} + \left(cn + ET\left(\frac{2n}{3}\right)\right) \cdot \frac{1}{3} + (cn + ET(n-1)) \cdot \frac{1}{3} \\ ET(n) &\leq cn + \frac{2}{3}ET(n-1) + \frac{1}{3}ET\left(\frac{2n}{3}\right) \\ ET(n) &\leq cn + \frac{2}{3}ET(n) + \frac{1}{3}ET\left(\frac{2n}{3}\right) \\ \frac{1}{3}ET(n) &\leq cn + \frac{1}{3}ET\left(\frac{2n}{3}\right) \\ ET(n) &\leq 3cn + ET\left(\frac{2n}{3}\right) \\ ET(n) &\leq c'n + ET\left(\frac{2n}{3}\right) \end{aligned}$$

Which after performing several substitutions obeys

$$ET(n) \leq \sum_{i=0}^{k-1} \frac{2^i c' n}{3^i} + ET\left(\frac{2^k n}{3^k}\right)$$

We choose k such that

$$\frac{2^k n}{3^k} = 1 \qquad \Longrightarrow \qquad k = \log_{3/2}(n)$$

Thus,

$$ET(n) \leq \sum_{i=0}^{k-1} \frac{2^i c' n}{3^i} + ET(1) \leq c'n \sum_{i=0}^{\infty} \left(\frac{2}{3}\right)^i + c \leq c'n \cdot \frac{1}{1 - \frac{2}{3}} + c$$

Therefore, $ET(n) \in O(n)$ and from the best case analysis performed earlier we know that $ET(n) \in \Omega(n)$. Thus,

$$\boxed{ET(n) \in \Theta(n).}$$